

目录

1. 芯片验证工程师应该具备的七种能力

1.1 RTL 设计理解

1.2 验证方法学

1.3 SoC 与协议

1.4 能不能理解系统级交互？从 block-level 到 SoC-level，验证难度会明显上升。原因不是模块数量简单增加，而是模块之间的交互开始变得复杂。AXI、PCIe、DDR、DMA、Interrupt、Coherency 这些内容，往往是系统级问题的高发区域。

1.5 软件与固件协同

1.6 平台与工具使用

1.7 能不能从发现 bug 走向风险闭环并自我成长

1.8 总结

2. 什么是上电复位POR以及为啥还需要专用的电源监控芯片？

2.1 POR（上电复位 Power-On Reset）

2.1.1 POR

2.1.2 内部POR潜在风险

2.2 STM32 RC外部复位电路

2.2.1 工作过程

2.2.2 RC两个核心作用

2.2.3 RC电路天生缺陷

2.3 为什么额外使用电源监控芯片（TPS3307这类）？

2.3.1 外部电源监控芯片的独有能力

2.3.2 使用场景

2.4 会不会出现无法捕获阈值、复位失效？

2.4.1 场景1：电压变化速率过快

2.4.2 场景2：迟滞电压（TPS3307自带，很多内置POR迟滞很小）

2.4.3 场景3：电源上电斜率过缓

2.5 数字系统电源与时序设计

2.5.1 步骤1：梳理所有电源轨，明确上电顺序要求

2.5.2 步骤2：明确需求清单

2.5.3 步骤3：复位方案选型对比

2.5.4 步骤4：风险预判

2.6 TPS3307总结

2.7 常见问题

3. 外设集成没人爱管，真正难的从来不是外设本身

3.1 真正难的从来不是外设本身

3.2 SoC 级验证该验什么，不该验什么

- 3.3 寄存器验证：最容易糊弄，也最容易反噬
 - 3.4 一点系统层面的想法
 - 4. 软件同事说程序跑到一半跳到了一个谁都不认识的地址
 - 4.1 验证环境该怎么搭
 - 5. 分层验证后仿思路
 - 6. PR后仿流程
 - 7. 什么是芯片 Bring-up
 - 7.1 Bring-up 发生在什么阶段？
 - 7.2 Bring-up 和其他验证活动有什么区别？
 - 7.3 Bring-up 通常怎样展开？
 - 7.3.1 确认电源和板级状态
 - 7.3.2 检查时钟和复位
 - 7.3.3 打通调试入口
 - 7.3.4 建立最小执行路径
 - 7.3.5 打通启动介质、DDR 和基础外设
 - 7.3.6 启动系统软件和关键数据路径
 - 7.4 为什么 Bring-up 难？
 - 7.5 Bring-up 的准备工作要在回片前完成
 - 7.5.1 可观测性
 - 7.5.2 可控制性
 - 7.5.3 测试与诊断能力
 - 7.6 FPGA Prototype 能提前暴露哪些问题？
 - 8. 【验证专项】寄存器专项测试
 - 8.1 寄存器复位值测试
 - 8.2 寄存器属性测试
 - 8.3 地址粘连测试
 - 8.4 数据粘连测试
 - 8.5 地址边界测试
 - 8.6 全地址遍历测试
 - 9. [验证专项][UPF 功耗仿真]UPF 功耗仿真原理通俗版讲解
 - 9.1 先搞懂普通仿真的问题
 - 9.2 UPF 核心作用：给仿真器加上「电源模型」
 - 9.3 第一步：划分电源区域 UPF 用 `create_power_domain` 把芯片分成好几块区域：
 - 9.4 功耗统计原理
 - 9.5 PA 功耗仿真整体运行机制
 - 9.6 一句话总结
-

1. 芯片验证工程师应该具备的七种能力

来源: <https://mp.weixin.qq.com/s/RPxiqCLEAMrG9tPhklb61w>

作者: 芯片验证

update 2026/08/16 01 : 12

很多人提到芯片验证工程师，第一反应是写 testbench、跑 simulation、看 waveform、收 coverage。这些当然是基本功，但不是全部。

随着 SoC、AI 芯片、Chiplet、HBM、高速接口和软件栈越来越复杂，验证工程师面对的问题已经不再局限于单个 RTL 模块。很多 bug 出现在设计、协议、软件、平台和系统场景的交界处。

所以，验证工程师的能力结构也需要从“单点技能”走向“系统能力”。一名优秀的芯片验证工程师应该具备以下7种能力：

芯片验证工程师能力图谱



一、数字硬件基础

数字硬件基础直接决定了能不能理解设计行为，是验证工程师最基础的能力。FSM、Clock、Reset、CDC、Timing 这些概念，看起来基础，但很多真实 bug 最后都会回到这些问题上。如果缺少数字硬件基础，验证工程师可能只能

看到 case fail，却很难判断问题到底是协议错误、状态机错误、时钟复位问题，还是环境建模问题。

1.1 RTL 设计理解

能不能把规格、波形和代码对应起来？验证工程师不一定每天写大量 RTL，但需要能读懂 RTL。这里的重点不是语法，而是理解设计结构。比如 CSR、FIFO、Pipeline、Arbiter、Error Path 这些模块，在真实设计中经常决定系统行为。

一个 DMA timeout，表面看是软件等不到完成信号；往下看，可能是 FIFO 没有出数，arbiter 没有授权，pipeline 中某一级 valid 没有传递，或者 error path 没有正确返回状态。这时验证工程师需要把几件事对应起来：

- 规格里说它应该怎么工作；
- 波形里实际发生了什么；
- RTL 代码为什么会走到这个状态？

1.2 验证方法学

能不能把风险变成可检查的工程任务？这考验的是验证手段，验证方法，验证效率，最终影响验证的质量。UVM、Assertion、Coverage、Regression 这些能力是验证工程师的核心专业能力。但它们的价值不在于“会不会用某个框架”，而在于能不能把设计风险拆解成：

- test plan；
- stimulus；
- checker；
- assertion；
- coverage point；
- regression case；
- Bug tracking

例如，验证一个 AXI 接口，不只是跑几个正常读写 case，而是要考虑 burst、alignment、outstanding transaction、backpressure、error response、reset in transaction 等场景。Coverage 也不是为了追求数字好看，而是为了回答：关键风险点有没有被系统性触达？

所以验证方法学体现的是：能不能把“不确定的设计风险”转化成可检查、可回归、可度量的验证任务。

1.3 SoC 与协议

1.4 能不能理解系统级交互？从 block-level 到 SoC-level，验证难度会明显上升。原因不是模块数量简单增加，而是模块之间的交互开始变得复杂。

AXI、PCIe、DDR、DMA、Interrupt、Coherency 这些内容，往往是系统级问题的高发区域。

例如：

- DMA 搬运数据，涉及 memory map、cache coherency、interrupt 和 driver；
- PCIe 枚举失败，可能和 reset、clock、BAR 配置、firmware 初始化有关；
- DDR 带宽不足，可能不只是 controller 的问题，也可能和 NoC arbitration、QoS、访问模式有关。

所以 SoC 与协议能力体现的是：能不能从单个模块的正确性，进一步理解系统交互是否成立？

1.5 软件与固件协同

现代芯片验证里，软件越来越早进入验证流程。Firmware、Linux、Driver、Log、Register Access，不只是软件团队的事情。很多硬件问题，只有在真实软件路径下才会被触发。需要理解软件如何触发硬件问题。

例如：

- bootloader 初始化寄存器顺序不符合 RTL 假设；
- driver enable interrupt 的时机暴露了 pending bit 问题；
- device tree 配置错误导致地址空间访问异常；
- firmware 对某个状态位的理解与设计实现不一致；
- Linux log 里的 timeout，其实指向硬件状态机没有响应。

验证工程师不需要变成软件工程师，但需要知道软件如何配置硬件、启动硬件、观察硬件。所以，软件与固件协同能力体现的是：能不能理解硬件问题如何被软件路径触发。

1.6 平台与工具使用

Simulation、Formal、Emulation、FPGA Prototype、Bring-up，每种工具和平台的使用对应不同层级的风险观察方式。

- RTL Simulation 适合协议细节、corner cases、assertions 和 coverage。
- Formal 适合状态空间、死锁、不变量和安全属性。
- Emulation 适合大规模 RTL、firmware、driver 和系统启动。
- FPGA Prototype 适合接近真实速度、真实 I/O、长时间 workload。
- Bring-up 面对真实硅片、真实板卡和真实物理环境。

所以平台与工具能力体现的是：能不能把不同验证平台连接成连续证据链，而不是孤立使用。

1.7 能不能从发现 bug 走向风险闭环并自我成长

系统工程能力是很多验证工程师后期成长的关键。Test Plan、RCA、Workload、Failure Mode、Risk Closure，这些能力决定了验证工作能不能从“发现一个 bug”走向“收敛一类风险”。例如，板子上发现一个 driver timeout，不能只修掉当前的 bug。还要继续追问：

- 这是单点问题，还是一类场景没有覆盖？
- 是否需要补 assertion？
- 是否需要补 coverage？
- 是否要把真实 driver sequence 抽象成 regression case？
- 是否暴露了 debug visibility 不足？
- 是否说明规格或架构假设需要调整？

所以系统工程能力体现的是：能不能把单个问题转化成可复用、可追踪、可闭环的工程经验。

遇到问题持续总结，学习，成长，将遇到过的问题沉淀成为自己的经验。以后再遇到类似的问题，可能就提前想到并处理，规避掉一些项目风险。这也是为什么工作经验值钱，期待每个验证工程师都能持续增值。

1.8 总结

芯片验证工程师的核心能力，不只是写 testbench，也不只是跑 simulation，而是把硬件、RTL、验证方法、SoC 协议、软件路径、平台工具和系统风险连接起来。

把复杂系统中的问题，转化成可检查、可复现、可回归、可闭环的工程证据。并实现自我成长，让自己升值。

2. 什么是上电复位POR以及为啥还需要专用的电源监控芯片？

来源: <https://mp.weixin.qq.com/s/6qkt7BFZPNdLi06R-mE9Fw>

作者: 嵌入式电子学习

update 2026/08/16 01:15

字数 3059, 阅读大约需 16 分钟

2.1 POR (上电复位 Power-On Reset)

2.1.1 POR

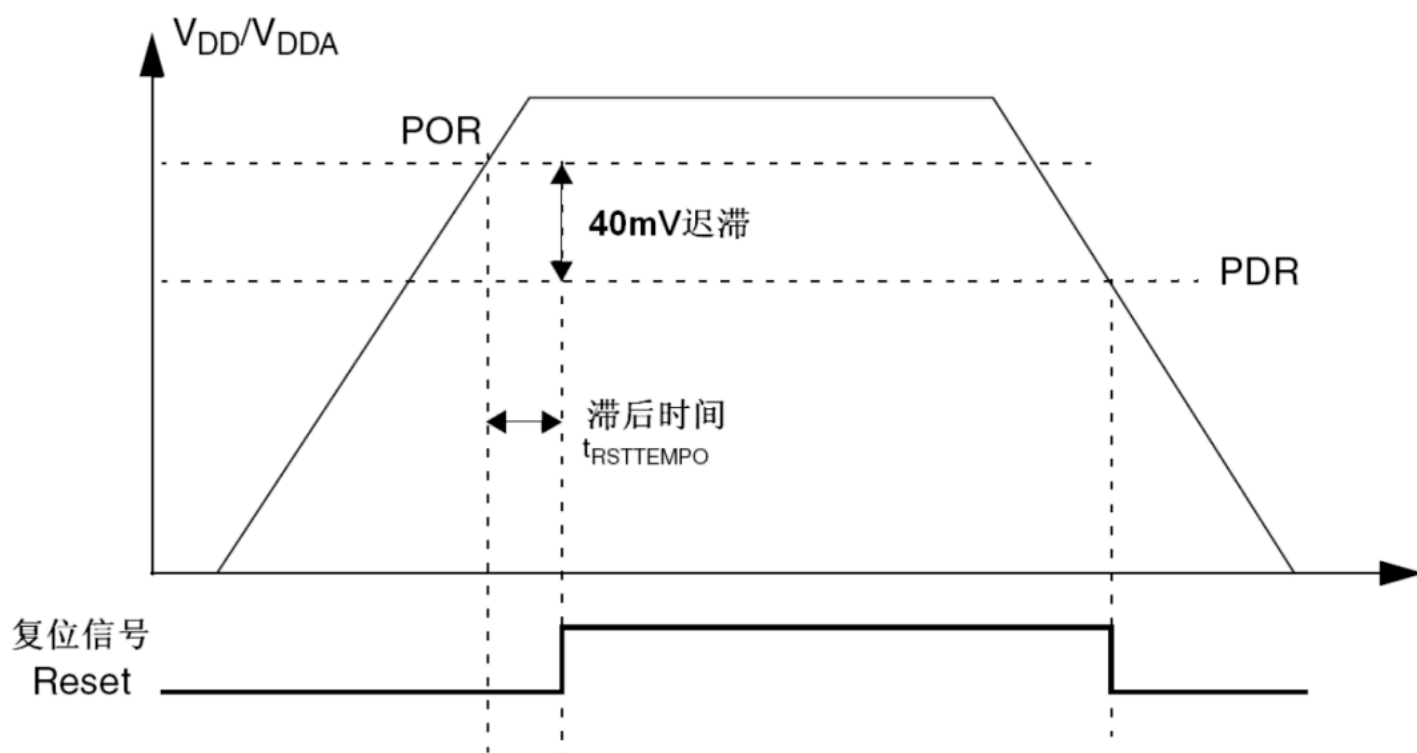
2.1.1.1 上电复位 Power-On Reset

- **本质目的：**电源电压从0慢慢爬升过程中，MCU/FPGA内核、寄存器、RAM状态是**随机不定态**。

- 电源刚上电，电压低于芯片最低工作电压时，逻辑单元电平不确定：寄存器可能随机0/1、总线冲突、外设乱启动、程序跑飞。
- 上电启动 (2)：上电复位

POR规则：

- 电源电压低于复位阈值 V_{th+} → 强制拉复位（芯片保持复位状态，不运行程序）
- 电压缓慢上升，超过阈值并且维持一段时间（对应POR输出的复位持续时间） → 释放复位，芯片开始正常启动、加载程序。
- 电源电压处于阈值电压以下时MCU一直处在复位状态！



2.1.1.2 下电复位（掉电复位BOR）

- **本质目的：**断电不是瞬间归零！很多场景电压缓慢跌落（电池耗尽、负载突变拉垮电源、插拔瞬间压降）。

如果电压慢慢掉到芯片最低工作电压之下，但**没有触发复位直接断电**：

- RAM、寄存器数据半截被改写、Flash正在写操作中途断电：**Flash数据损坏**；
- 下次上电芯片状态混乱，无法正常启动。

BOR/下电POR：电压下跌低于**负阈值 V_{th-}** ，立刻再次触发复位，**让芯片进入安全复位状态，防止器件在亚稳态区间无序工作。**

简单总结：
上电复位：等电压足够高，再开机
下电复位：电压不够高的时候，立刻强制停机

2.1.2 内部POR潜在风险

内部POR是芯片内置模拟电路，天生有短板：

2.1.2.1 阈值精度差

- STM32、FPGA内置POR阈值是范围值，比如内置POR阈值可能1.8~2.4V区间浮动，批次差异、温度漂移大。

2.1.2.2 没有可调延时

- 内置POR固定复位保持时间，不受外部控制。
- 系统有多芯片（MCU+FPGA+DDR+PHY）时，电源先后上电、各芯片就绪时间不一致，时序无法匹配。

$V_{PVDhyst}^{(2)}$	PVD迟滞			100		mV
$V_{POR/PDR}$	上电/掉电复位阈值	下降沿	1.8 ⁽¹⁾	1.88	1.96	V
		上升沿	1.84	1.92	2.0	V
$V_{PDRhyst}^{(2)}$	PDR迟滞			40		mV
$T_{RSTTEMPO}^{(2)}$	复位持续时间		1	2.5	4.5	ms

2.1.2.3 抗干扰弱

电源存在尖峰、纹波、瞬时跌落：

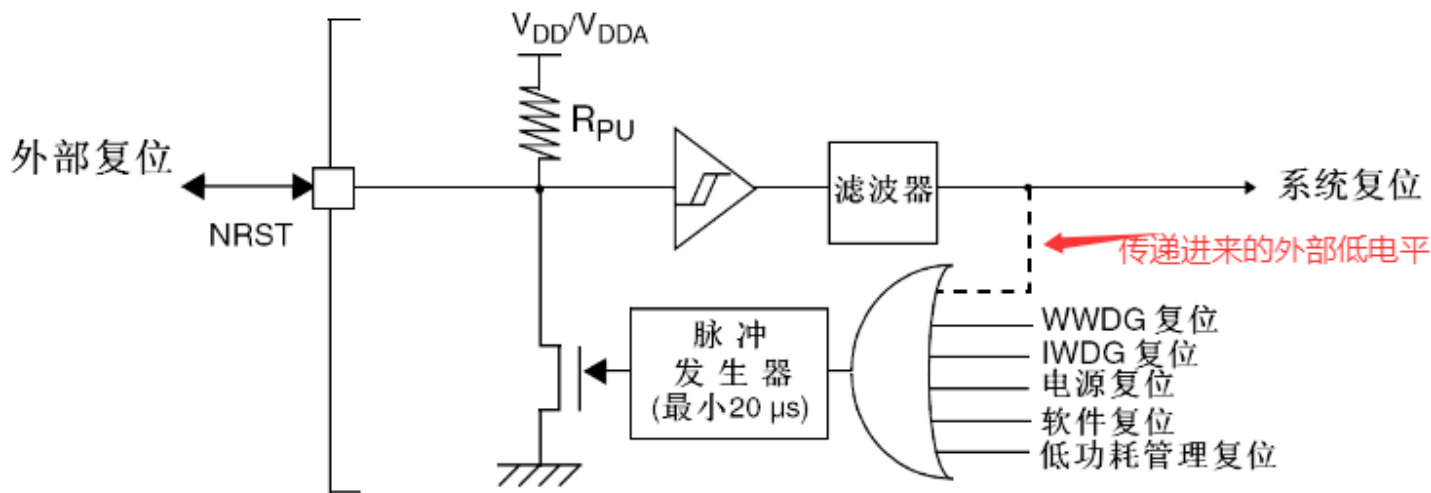
- 瞬时电压短暂低于阈值又回升（迟滞区间不足）→ 内部POR可能误触发复位；
- 或者窄脉冲电压扰动时间太短，POR内部电路来不及响应 → 【最关键问题：漏复位！】

2.1.2.3.1 会不会捕获不到阈值，导致无法复位？

- POR模拟电路需要建立时间。如果电压快速跌落/快速上升，电压跨越阈值的持续时间极短，内部比较器来不及采样识别，复位信号不会产生。
- 典型场景：热插拔、电源瞬间抖动、浪涌、负载突增产生的短暂电压凹陷。

2.1.2.4 监控电压单一

- 芯片内部POR只监控主内核供电（比如3.3V）。
- 系统有多路电源：1.0V内核、1.8V IO、2.5V PLL、DDR电源。内部POR看不到其他轨电压。某一路辅助电源失效，主电源正常，内部POR不会触发复位，系统照样异常。

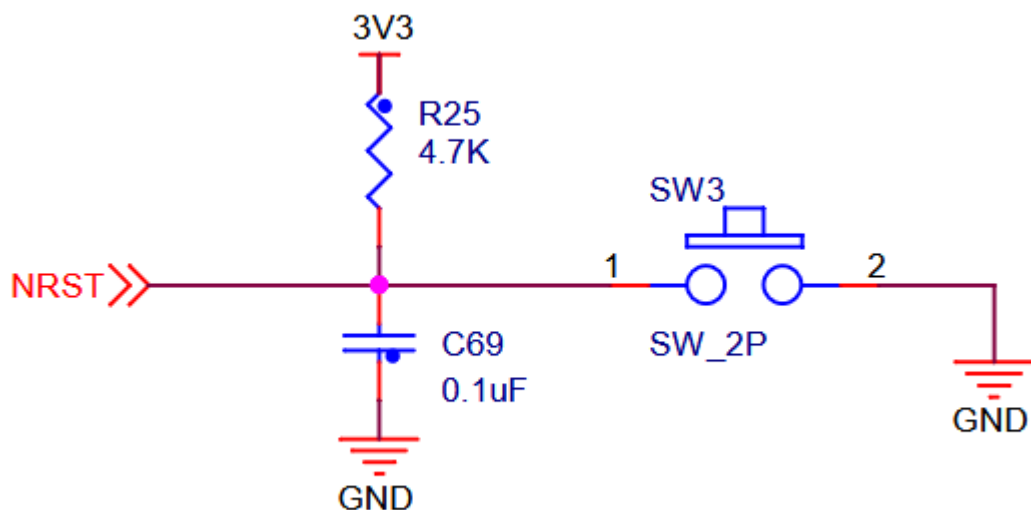


2.1.2.5 无法多器件同步复位

- MCU内部POR只管自己，FPGA内部POR只管自己。两者上电快慢不同，一个先启动、一个后启动，通信时序错乱。

2.2 STM32 RC外部复位电路

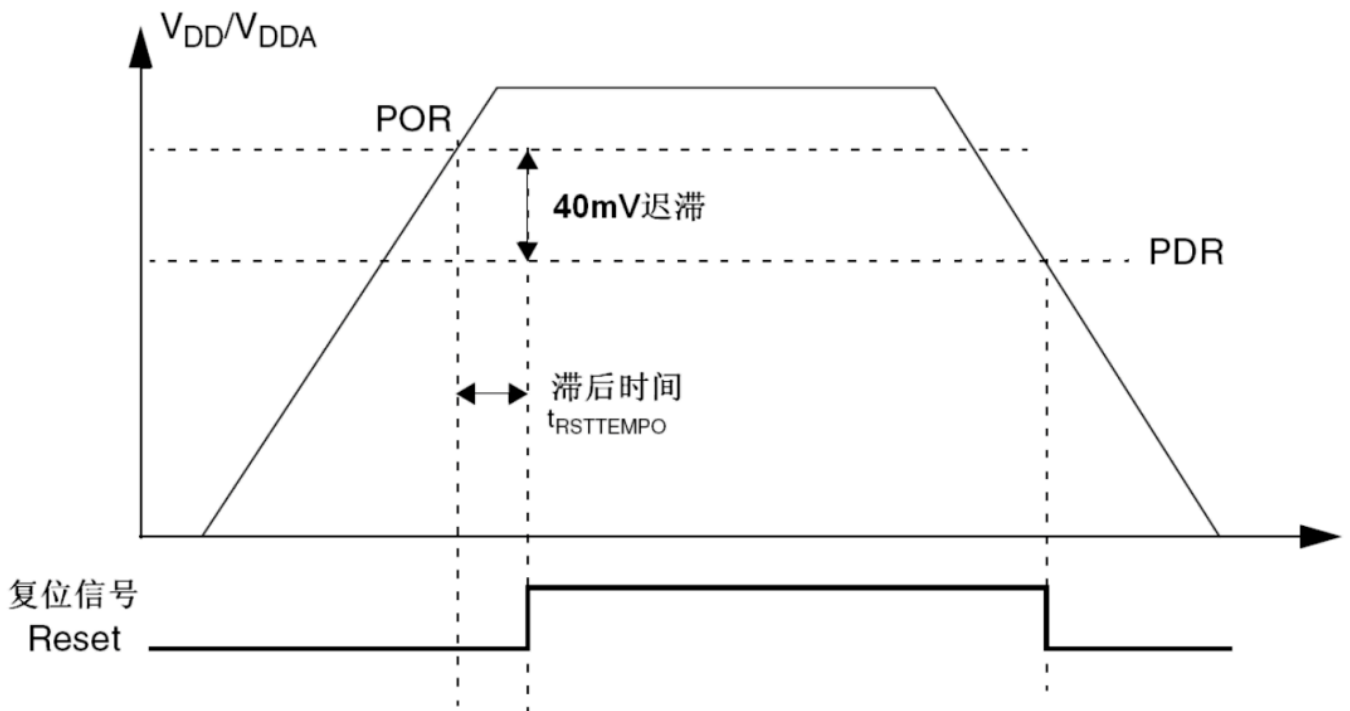
经典电路：NRST引脚 → 上拉电阻R接到3.3V；电容C接到GND



2.2.1 工作过程

- 上电瞬间：**电容两端电压差值不能突变。上电时刻3.3V建立，电容初始电压=0V，NRST被电容拉低 → **持续复位**。
- 电源稳定后，3.3V通过电阻R给电容C充电，电容电压指数上升；

3. 当NRST引脚电压上升超过芯片**复位释放阈值**，复位解除，STM32启动。



2.2.2 RC两个核心作用

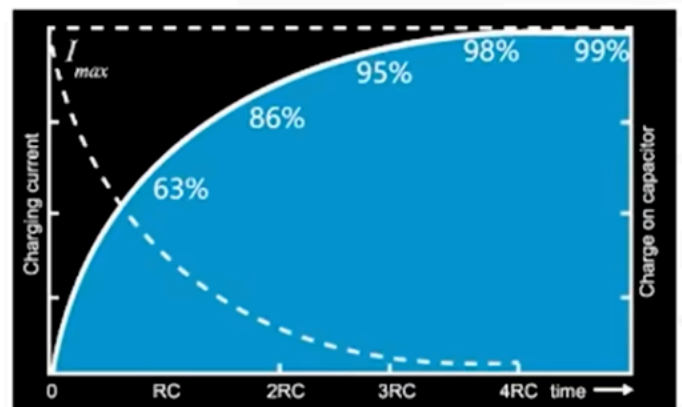
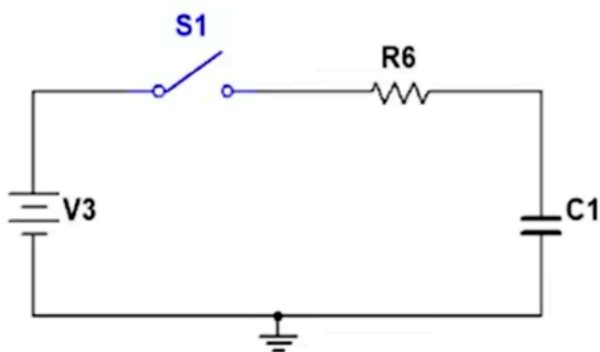
2.2.2.1 产生复位延时（首要目的！）

- 通过RC充电时间常数 $\tau = R \cdot C$ ，人为拉长**低电平复位持续时间**。
- 内置POR延时很短，若电源上升斜率很陡，内置复位时间不够，RC延长复位保持时间，保证内部电路稳定后再启动。

电容可以简单理解成一个小电池

电容的充电速度与电容大小以及充电电流有关

电容的充电时间常数： $\tau = R \cdot C$



2.2.2.2 滤除高频干扰（次要）

- 电源上电尖峰、空间电磁干扰，容易在NRST引脚上产生窄尖峰脉冲。电容抑制高频毛刺，防止**误复位**。

2.2.2.3 常见误区纠正

- 错误：RC只是滤波
- 真相：**延时是核心，滤波是附加收益**

2.2.3 RC电路天生缺陷

1. 复位阈值完全由芯片内部决定，外部无法调整；
2. **无法监控电源电压！** RC电路只看上电时序，电压跌到2V还是3V，RC完全无感。电压跌落时RC不会自动产生复位；RC复位**只有上电复位能力，没有掉电监控BOR功能！**电源缓慢掉电场景完全失效；
3. 温漂、电容老化会改变延时；
4. 无法应对电压短暂跌落（电压掉下去又快速回升，RC不会触发二次复位）；
5. 不能同步给FPGA、其他外设提供统一复位信号。

结论：RC复位=低成本简易方案，适合小家电、简单单片机；复杂工业设备、带FPGA、高速接口、DDR的系统不够可靠。

2.3 为什么额外使用电源监控芯片（TPS3307这类）？

- TPS3307是典型**多路电压监控+独立POR+推挽复位输出**电源监控IC。
- **不是内部POR完全不能用，是内部POR满足不了复杂数字系统的时序与可靠性要求。**

2.3.1 外部电源监控芯片的独有能力

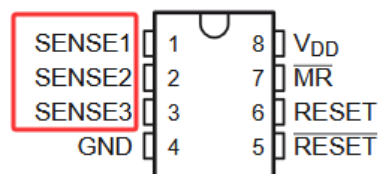
2.3.1.1 精准、稳定的电压检测阈值

- TPS3307阈值由芯片内部带隙基准决定，精度高、温漂小。
- STM32内置POR阈值误差±几百mV，工业高低温环境下波动巨大。
- 可以设定：3.3V电源低于2.9V立刻触发复位，阈值严格可控。

FEATURES

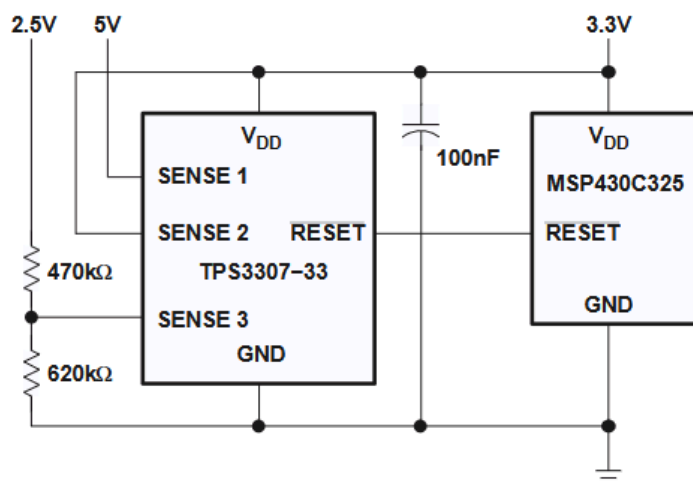
- Triple Supervisory Circuits for DSP and Processor-Based Systems
- **Power-On Reset Generator With Fixed Delay Time of 200ms, No External Capacitor Needed**
- Temperature-Compensated Voltage Reference
- Maximum Supply Current of 40 μ A
- Supply Voltage Range: 2V to 6V
- Defined **RESET** Output From **$V_{DD} \geq 1.1V$**
- MSOP-8 and SO-8 Packages
- Temperature Range : - 40°C to +85°C

D OR DGN PACKAGE
(TOP VIEW)



TYPICAL APPLICATIONS

[Figure 1](#) lists some of the typical applications for the TPS3307 family, and a schematic diagram for a processor-based system application. This application uses TI part numbers [TPS3307-33](#) and [MSP430C325](#).



- Applications using DSPs, Microcontrollers or Microprocessors
- Industrial Equipment
- Programmable Controls
- Automotive Systems
- Portable/Battery Powered Equipment
- Intelligent Instruments
- Wireless Communication Systems
- Notebook/Desktop Computers

Figure 1. Applications Using the TPS3307 Family

2.3.1.2 独立可编程复位延时（解决多芯片时序）

- 监控芯片上电后，电压达标，还会维持一段固定复位时间 t_{RESET} ，延时可选（ms级别）。

工程价值：

- 系统存在多级电源：先产生5V → 转换成3.3V → 1.8V → 1.0V内核电压。
- 各路电源建立有先后顺序。

需要：所有电源轨全部稳定就绪之后，再释放全局复位。

- MCU/FPGA不能某一路电源还没稳定就启动。
- 内置POR只能等待自身电压，不知道其他电源状态。
- 外部监控芯片可以配合多路监控，保证“全部电源就绪再开机”。

During power-on, $\overline{\text{RESET}}$ is asserted when the supply voltage V_{DD} becomes higher than 1.1V. Thereafter, the supply voltage supervisor monitors the SENSE_n inputs and keeps $\overline{\text{RESET}}$ active as long as SENSE_n remain below the threshold voltage $V_{\text{IT+}}$.
VDD=1.1v即可工作输出复位有效

An internal timer delays the return of the $\overline{\text{RESET}}$ output to the inactive state (high) to ensure proper system reset. The delay time, $t_{\text{d (typ)}} = 200\text{ms}$, starts after all SENSE_n inputs have risen above the threshold voltage $V_{\text{IT+}}$. When the voltage at any SENSE input drops below the threshold voltage $V_{\text{IT-}}$, the $\overline{\text{RESET}}$ output becomes active (low) again.
所有监控电压上升到阈值以上后持续输出200ms有效复位，然后复位才无效

The TPS3307-xx family of devices incorporates a manual reset input, $\overline{\text{MR}}$. A low level at $\overline{\text{MR}}$ causes $\overline{\text{RESET}}$ to become active. In addition to the active-low $\overline{\text{RESET}}$ output, the TPS3307-xx family includes an active-high RESET output.
MR有效也可以造成输出有效复位

The devices are available in either 8-pin MSOP or standard 8-pin SO packages.

The TPS3307-xx devices are characterized for operation over a temperature range of -40°C to $+85^{\circ}\text{C}$.

SUPPLY VOLTAGE MONITORING 提供三个监控引脚，内部比较阈值固定

DEVICE	NOMINAL SUPERVISED VOLTAGE			THRESHOLD VOLTAGE (TYP)		
	SENSE1	SENSE2	SENSE3	SENSE1	SENSE2	SENSE3
TPS3307-18	3.3V	1.8V	User defined	2.93V	1.68V	1.25V ⁽¹⁾
TPS3307-25	3.3V	2.5V	User defined	2.93V	2.25V	1.25V ⁽¹⁾
TPS3307-33	5V	3.3V	User defined	4.55V	2.93V	1.25V ⁽¹⁾

(1) The actual sense voltage has to be adjusted by an external resistor divider according to the application requirements.

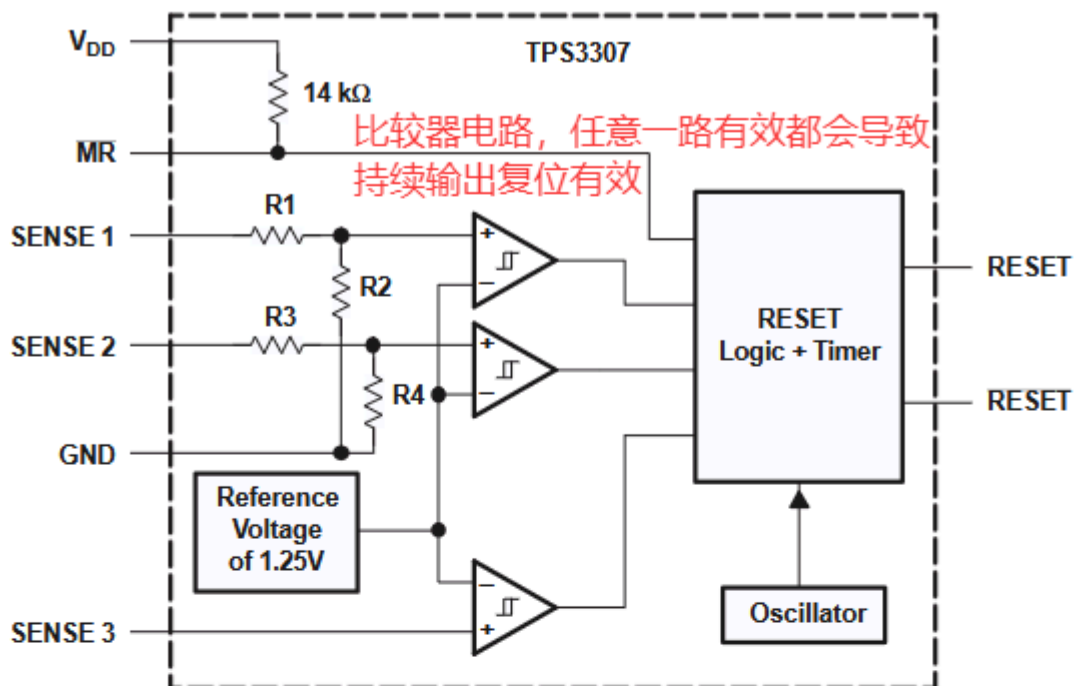
2.3.1.3 同时实现【上电复位 + 掉电复位 (BOR) 】

- RC电路没有掉电检测！电压慢慢往下掉的时候无反应（除非MCU内部自带BOR）。

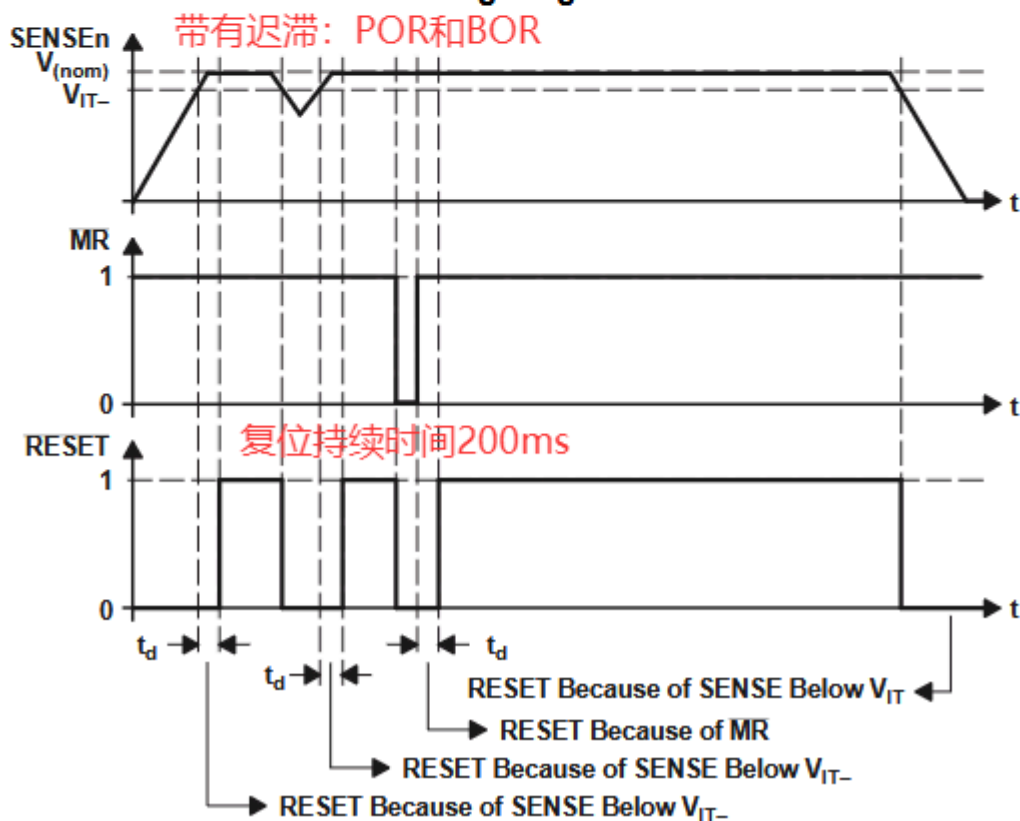
TPS3307持续实时监测电源：

- 电压高于阈值 → 复位释放
- 电压一旦跌破阈值 → 立刻拉低复位，无论上电还是运行中跌落。
- 完美解决电压跌落、负载扰动导致的亚稳态运行。

Functional Block Diagram



Timing Diagram



2.3.1.4 全局统一复位源 (MCU+FPGA+DDR+PHY时序同步)

这是大型硬件系统最核心需求：

- MCU有自己内部POR、FPGA也有内置POR。两者上电启动时间天生不同。

如果各自独立复位：

- FPGA先启动，等待MCU；
- 或者MCU先运行，FPGA还在复位，双方通信出错、FIFO溢出、总线异常。

解决方案：

- 使用TPS3307单一复位信号，同时驱动MCU NRST、FPGA nCONFIG、DDR复位引脚。
- 所有器件同时进入复位，同时释放复位，全局时序对齐。

2.3.1.5 多电源轨监控（TPS3307可同时监控2路/3路电压）

- FPGA系统常见：3.3V IO、1.2V内核、2.5V 辅助电源。
- 只要任意一路电压异常，立刻触发全局复位。
- 芯片内置POR只能监测单一供电，别的电源失效无法感知。

Function/Truth Tables

MR	SENSE1 > V _{IT1}	SENSE2 > V _{IT2}	SENSE3 > V _{IT3}	RESET	RESET
L	X ⁽¹⁾	X ⁽¹⁾	X	L	H
H	0	0	0	L	H
H	0	0	1	L	H
H	0	1	0	L	H
H	0	1	1	L	H
H	1	0	0	L	H
H	1	0	1	L	H
H	1	1	0	L	H
H	1	1	1	H	L

(1) X = Don't care

只有这个情况下输出复位无效

2.3.1.6 强驱动复位信号，抗干扰

- 内置POR复位信号在芯片内部，外部引脚驱动能力有限；
- TPS3307推挽输出复位信号，可以驱动多个芯片复位引脚，不容易被干扰拉高/拉低；还能配合增加噪声滤波。

2.3.2 使用场景

- 低成本消费类简单MCU产品：单电源、无高速外设、无FPGA、无DDR → STM32内置POR + RC复位足够。
- 工业控制、通信设备、FPGA+MCU架构、带DDR、要求高稳定性、宽温工作、电源环境恶劣：必须外部电源监控芯片。

2.4 会不会出现无法捕获阈值、复位失效？

2.4.1 场景1：电压变化速率过快

- POR/BOR比较器需要一定响应时间。

- 电压快速跌落，短暂低于阈值几微秒立刻恢复；
- 内部模拟电路来不及响应 → **不产生复位，漏检。**

外部TPS3307同样有响应时间，但可以搭配：

- 合适的延时、输入端RC滤波，过滤短暂毛刺，同时保证持续电压跌落可靠识别。

2.4.2 场景2：迟滞电压（TPS3307自带，很多内置POR迟滞很小）

- 如果没有电压迟滞：电压在阈值附近小幅来回波动，复位反复震荡（不断复位、释放、复位）。

芯片设计增加迟滞：

- 上升阈值 V_+ ，下降阈值 V_- （ $V_+ > V_-$ ）。
- TPS3307内置迟滞，避免阈值附近震荡；很多MCU内置POR迟滞宽度小，容易出现反复抖动。

2.4.3 场景3：电源上电斜率过缓

- 电压极其缓慢爬升，长时间徘徊在阈值附近，容易让内部模拟电路进入亚稳态，复位逻辑异常。外部监控芯片时序特性更稳定。

2.5 数字系统电源与时序设计

2.5.1 步骤1：梳理所有电源轨，明确上电顺序要求

- 哪些电源需要先建立？
- 内核电压还是IO电压？
- FPGA通常要求内核先于IO上电。

2.5.2 步骤2：明确需求清单

1. 是否需要上电复位？
2. 是否需要运行中电压跌落检测（BOR掉电复位）？
3. 芯片数量：是否同时存在MCU、FPGA、DDR、以太网PHY，是否需要**同步全局复位**？
4. 环境：常温消费产品 / 工业宽温、强电磁干扰？
5. 失效后果：只是重启，还是会造成设备误动作、数据损坏？

2.5.3 步骤3：复位方案选型对比

1. 仅芯片内置POR/BOR：最便宜，可靠性下限最低，无法多芯片同步，无多路电源监控
2. 内置POR + 外部RC复位：增加上电延时，无掉电监控，低成本折中方案

3. 专用电源监控芯片（TPS3307、MAX809、ADM705）：精准阈值、上电+掉电监控、可编程延时、全局复位输出，工业主流方案

2.5.4 步骤4：风险预判

- 电压抖动会不会漏复位？增加输入端RC滤波；
- 复位信号布线容易受干扰？复位线路短、远离高速信号线；
- 多个器件复位电平（高有效/低有效）不一致，增加电平转换；
- 上电延时是否满足所有电源稳定时间，预留裕量。

2.6 TPS3307总结

- TPS3307本质就是一块**独立、高精度、带时间延迟的电压比较器系统**。
- 它不和MCU/FPGA内部电路绑定，独立持续监测供电电压；一旦电压不合格，强制输出复位电平；电压达标后，等待预设时间再放开复位。
- 作用就是把“**电源合格**”作为所有数字芯片启动的前置必要条件，统一所有器件启动时序，弥补MCU/FPGA内置POR精度不足、监控范围有限、无法多芯片同步的短板。

2.7 常见问题

1. RC复位电路不要电容选太大！电容过大，上电复位时间太长；
2. 使用外部电源监控芯片时，MCU内部POR依然可以保留，形成双重保护；
3. FPGA特别依赖稳定复位，FPGA配置芯片、nCONFIG信号时序出错会导致无法加载程序，复杂FPGA项目几乎都会搭配外部电源监控。

3. 外设集成没人爱管，真正难的从来不是外设本身

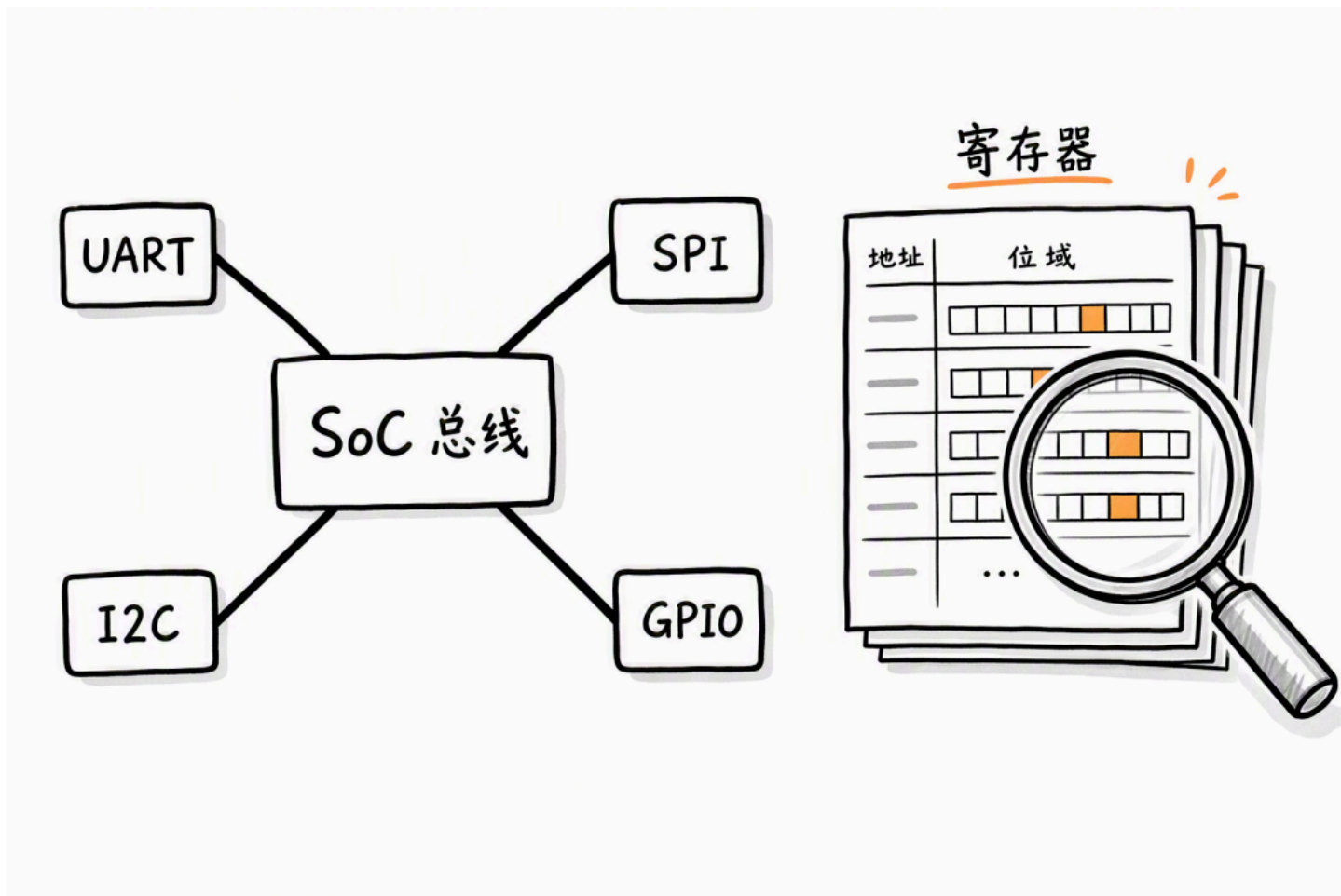
来源：<https://mp.weixin.qq.com/s/SC7SO59zxi-vbK6bDqTOQg>

作者：make ic

update 2026/08/16 03 : 03

3.1 真正难的从来不是外设本身

UART 有多难？波特率分频、起始位、停止位、奇偶校验，一个星期能验完。SPI、I2C、Timer、PWM 也一样，都是几十年的老协议，IP 厂商卖了上千次，内部 bug 早被市场清洗干净了。



但每次 SoC 流片回来，出问题的清单上总有它们的名字。

外设 IP 级验证和 SoC 级验证，验的是两件不同的事。IP 级验的是协议行为，SoC 级验的是这个 IP 在这颗芯片里的连接关系、地址映射、时钟复位、中断路径、复用配置、低功耗行为。IP 厂商保证前者，后者只属于你。

工程上真正容易出错的地方，永远是接缝，而不是零件。

3.2 SoC 级验证该验什么，不该验什么

SoC 级别再去跑一遍 UART 的协议一致性，是在浪费机器时间。

SoC 级该盯的是这几件事：

- 地址译码有没有错，两个外设有没有踩到同一段空间
- 中断线接对了没有，PWM 的 fault 是不是接到了别人的输入
- 复位域交叉，异步复位释放顺序
- pinmux 复用后，SPI 的 MISO 是不是被 GPIO 抢走了

这些都不需要复杂的 sequence，需要的是**连接性和配置的全覆盖**。用连线检查（connectivity check）和形式化工具做，比写一堆定向用例靠谱得多，也快得多。

集成阶段最常见的翻车现场：某个信号在顶层被接成常量 0，仿真里毫无异常，因为没人激励过它。等到 FPGA 上板才发现，中断永远不来。

3.3 寄存器验证：最容易糊弄，也最容易反噬

寄存器验证有个特点，看起来最简单，因此最容易被草率对待。

标准做法是 UVM RAL，跑 `uvm_reg_hw_reset_seq`、`uvm_reg_bit_bash_seq`、`uvm_reg_access_seq`。三个 sequence 跑过，报告一片绿，sign off。

问题出在这里：**这三个 sequence 只能证明"寄存器模型和 RTL 一致"，证明不了"寄存器模型和 spec 一致"。**

如果 RAL 模型是从 RTL 反推手写的，那这个验证就变成了自己和自己对账。

唯一可靠的解法是**单一数据源**：spec (IP-XACT 或者 excel/yaml) 自动生成 RTL、RAL 模型、C header、文档，四份产出同源。这从根上消除人工同步这个错误来源。这条路业界走了很多年，TI 在 DVCon 上就分享过 IP-XACT 到 UVM 寄存器模型的完整实践。

还有几类寄存器，标准 sequence 基本无能为力，得手工补：

```
// 只读状态位、W1C 中断标志、shadow/lock 寄存器
```

```
// 写保护寄存器、side-effect 寄存器（读一下就清）
```

```
// 这些必须 set_compare(UVM_NO_CHECK) 后单独写用例
```

尤其是 **W1C**（写 1 清零）的中断标志。它的行为依赖硬件的置位时机，标准访问测试跑过去只会得到一堆假通过。

另外 SoC 级用 RAL 有个现实：RAL 走的是前门访问，路径长、地址转换多，全片寄存器扫一遍可能要几小时。合理做法是 SoC 级只抽查关键寄存器和地址可达性，把全量 bit bash 留在 IP/子系统级完成。

3.4 一点系统层面的想法

外设验证的地位很尴尬。它不出彩，写在周报里没人夸；它出事，责任又落不到具体某个人身上，因为"IP 是买的""连线是集成组接的""寄存器表是设计给的"。

验证工程师真正的价值，恰恰在这些没有明确责任人的地方。

复杂系统的失效，很少来自某个模块被做得不够好，多数来自模块之间的假设没有对齐。DV 的职责就是把这些隐含假设逼出来，写成可执行的检查，让它在仿真里而不是在客户手里暴露。

至于"外设很简单"——它通常出自还没有被 pinmux 折磨过的人。

4. 软件同事说程序跑到一半跳到了一个谁都不认识的地址

来源: https://mp.weixin.qq.com/s/YMH_RcKZmfJ17xBkOoGihQ

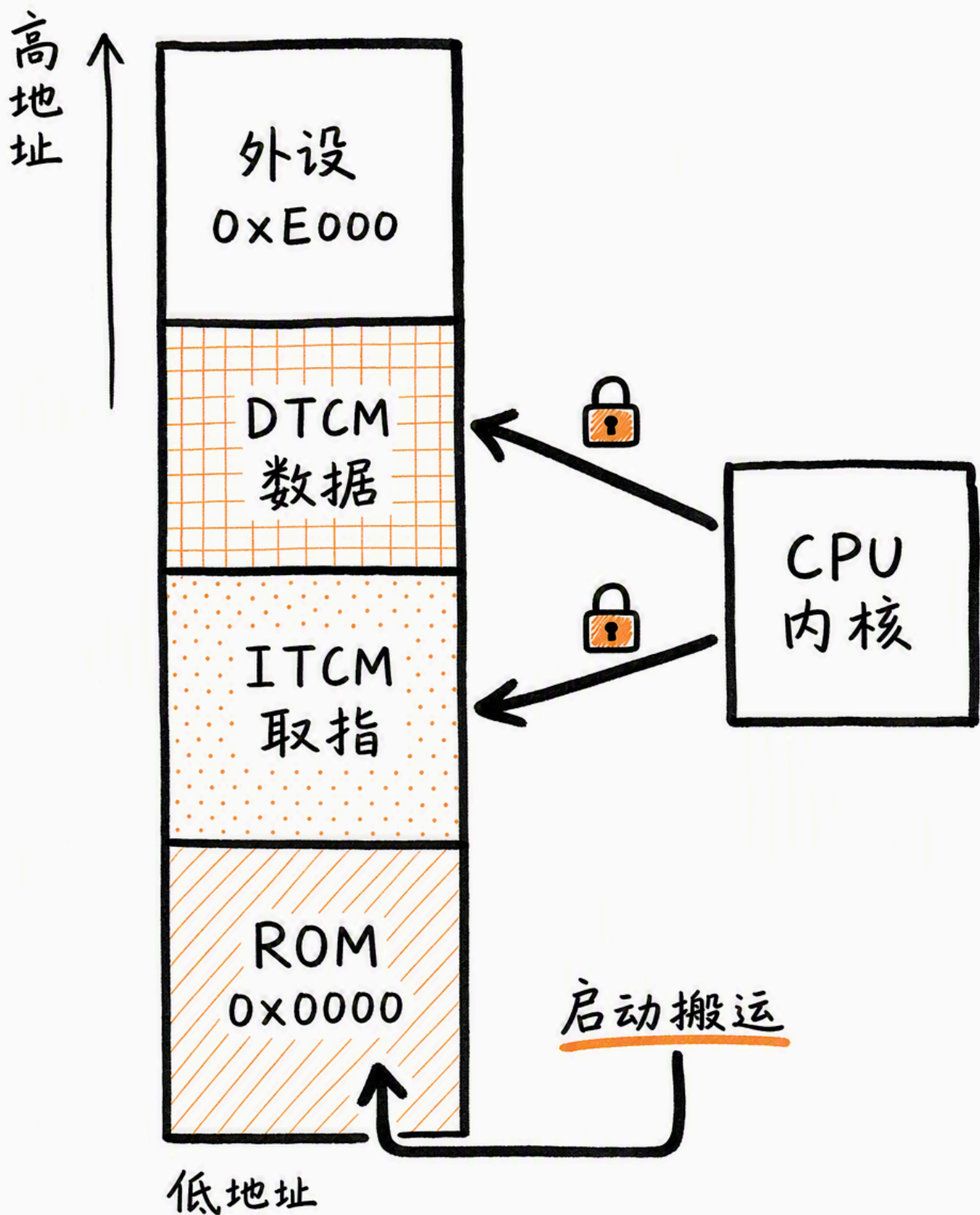
作者: make ic

update 2026/08/16 03 : 06

芯片回来了，软件同事说程序跑到一半跳到了一个谁都不认识的地址。

翻波形，翻代码，翻文档，最后发现问题出在一段**没人验证过的地址空间**——ITCM 的尾部越界之后，总线既没有报错，也没有返回数据，直接把 CPU 挂在那里等一个永远不会来的 ready。

这类问题不稀奇。SOC内存子系统的验证，最容易被当成"接口通了就算过"。



处理器发出的地址只是一串二进制位。它没有语义，没有属性，也不知道自己该去哪。

是译码逻辑赋予它意义。

ITCM、DTCM、ROM、外设寄存器，这些名字全部活在文档和译码器的比较逻辑里。

所以内存映射验证的第一个动作，从来不是跑一段程序看能不能启动，而是回答三个问题：

一、每一个地址落在哪个区间？

二、落在两个区间之间的空洞里会怎样？

三、落在区间里但权限不符合会怎样？

第三个问题最容易被跳过。

上电那一刻，TCM 里是随机值或者全零。CPU 必须从 ROM 取第一条指令。

搬运程序的这段代码，自己也得有地方跑。

这里有个经典的死锁式设计问题：**搬运 ITCM 的循环，能不能自己待在 ITCM 里？**显然不能。它必须留在 ROM，还要在搬完之后小心地跳过去。而“跳过去”的那一拍，PC 值从 ROM 空间切到 ITCM 空间，流水线里可能还残留着按旧映射预取的指令。

4.1 验证环境该怎么搭

别用 CPU 的行为反推内存系统对不对。

CPU 是个太善良的测试者。它只会发出编译器生成的那些访问，规规矩矩，对齐良好。真实世界里的问题来自 DMA 抢占、非对齐访问、跨边界突发、以及调试器在 CPU 停住时的旁路读写。

这些都需要一个**独立的 bus agent** 直接怼在总线上，绕开处理器，按地址空间逐段扫描。扫描的目标是建立一张实际映射表，然后和文档里那张表做逐位对比。两张表不一致的地方，就是bug。

商业上也能看出内存子系统的分量。近两年在 AI 加速器和边缘推理芯片里，紧耦合存储的容量比例明显上升，因为搬数据的功耗已经超过算数据。这意味着 TCM 相关的设计只会更复杂，多 bank、多端口、带 ECC、带低功耗保持，每加一项，映射和权限的组合就翻一倍。

内存映射是芯片里少数**完全由人定义、没有物理约束的部分**。想怎么划就怎么划。

正因为自由，它才容易错。

物理上不成立的东西还有EDA工具拦住，逻辑上不合理的地址划分，没有任何工具会替你把关。

一份靠谱的内存子系统验证，最终交付的东西是一张经过实测确认的地址表。它比任何覆盖率数字都有说服力，因为它回答的是软件同事真正会问的那个问题：这个地址，我能不能碰。

5. 分层验证后仿思路

来源: <https://mp.weixin.qq.com/s/wljtzKYp8Rg9R1lq3lnN-A>

update 2026/08/16 03 : 29

不像RTL功能仿真，后仿，特别是带SDF的PR网表仿真，我们很难避免的问题就是仿真效率。大型AI芯片，动辄几百亿门的规模，想让后仿跑起来，并能充分验证，达到signoff的标准，就不得不采取层次化后仿的思路。

从RTL功能仿真（前仿）的层次划分来看，有UT、IT、ST的验证分类，那么最朴素的想法，我们也按照UT、IT、ST的层次划分对应去做后仿真。是否可行呢，需要先澄清下面几个问题：

- 1.为什么我们最先想要按照前仿的划分来做？这个很朴素，前仿环境可以直接复用，用例可能也不需要怎么修改就可直接复用，这样对后仿验证来说，工作量最小。很直观的理由，但不一定可行。
- 2.前仿层次和后仿层次是否对应？以目前的项目经验来看，后仿的harden层次结构和前仿层次差异很大，且基本上无法对齐。这主要取决于以下几点：前仿UT、IT的分割主要基于功能完整性、验证可行性的角度进行分割；而后仿harden层次结构的分割主要受到后端时序、面积、布局布线的影响，某一harden可能包含多个UT，甚至只包含其中的某一部分。这带来的问题是，除非你基于harden层级再重做一套匹配的验证环境，测试用例，否则基于harden层次的后仿验证就无法开展。
- 3.可否基于ST网表来做signoff？显然不行，仿真效率慢的无法接受。目前项目中，一个冒烟soc前仿用例时间（不做任何后门加速）可能需要2天左右，以最保守的时间来评估，那后仿也需要40天左右的时间。这还只是一条用例，想做完整的signoff，整体时间无法接受。

既然存在上述的一些问题，那么后仿可选的方案就很有限了：要么基于UT、IT层次的网表来做；要么就仅基于IT层次的网表来做。

这里分析一下是否基于UT层次做后仿这件事。首先上面提到的1. 后端不会提供每个前仿UT的harden网表给你去做后仿。2. 全片接近100个UT，全做UT后仿，人力资源也吃不消。3. UT后仿的大部分场景可以直接在IT后仿中进行覆盖，且时间可接受。

倒是也不是说UT就完全不用考虑，对于一些复杂的异步、数模混合模块，在有时间、人力的情况下也可以考虑做一做。

那么现在可选的也只剩用IT层次网表来做后仿了。但也存在几个问题需要解决：

1.IT前仿环境和IT harden层次（soc-1层）网表不匹配，无法直接复用。针对这种情况，我们选择直接在soc rtl中，替换部分harden网表，其他harden依旧是rtl代码。这样，使用soc环境，通过libmap即可完成环境适配，仿真时间也在可接受的范围内。

2.那如果我们刚好有和harden层级匹配的IT前仿环境呢？显然直接使用IT前仿环境是更好的选择，毕竟通过soc部分替换的策略，只是因为harden网表不匹配的无赖之举。

3.现在所有soc-1的harden层级都有了其验证归属，那soc呢，难道就真的不需要了么？也没那么绝对，soc全片的时钟、复位、低功耗绝对是后仿的重点内容，这些内容在soc-1的harden层级中很难验证到。所以，如果外部条件允许，我们需要裁剪一个后端能做的、后仿能跑的soc简化架构，保留时钟、复位主体框架，并在此网表下做基础功能验证。

6. PR后仿流程

来源: <https://mp.weixin.qq.com/s/8eNXre6rw-BQUHNCY5WVLA>

update 2026/08/16 03 : 38

本文谈一下PR网表（带sdf反标）的后仿流程。

1.网表文件准备。设计提供给你的PR网表压缩包，打开看看，和设计确认一下网表类型，是PR网表、还是PR网表+UPF、还是PG网表。一般说来，单纯功能仿真使用PR网表就可以，如果涉及到不同电源域的低功耗验证，就需要PR网表+UPF，如果需要功耗验证，最好选择PG网表。

2.三级同步器第一级触发器的屏蔽文件准备。第一级触发器在后仿过程中，一定是报时序违例，但它是无害的，需要屏蔽，否则影响timing violation检查与收敛。具体操作方法就是让ai写个脚本，帮你提取网表文件中所有三级触发器的（带dff3后缀）第一级触发器的实例路径。然后基于vcs支持的格式 `instance {path}{noTiming};` 生成屏蔽文件，后面仿真脚本中会用。

3.Sram复位。Sram不是仿真加载进去后就可以用的，需要通过force的方式，在仿真前进行复位与解复位。方法和触发器屏蔽类似，让ai写个脚本，提取网表文件中所有sram的实例路径（一般sram命名都是sadr\sasr\sass等开头，看看sram命名就知道了），force sram的RSCRST信号，先force为0，再force为1，再release。有时候在网表中RSCRST信号会和其他信号关联起来，这个时候就不能force sram接口上的RSCRST，就需要往sram中找一级，force它。

4.SDF文件准备。先确认后仿signoff的corner选择，一般选择三个corner，TT、SS、FF，对应选择不同工艺、温度、电压。这块是和后端一起确认的，后仿过程中，只需要打开SDF文件，在文件开头部分，会提供当前SDF的corner信息，确认与约定的corner参数一致。

5.单元库的准备。分为两个部分，1. 各种非sram的cell，以及sram。其中非sram的cell，主要包括各种类型的逻辑门、以及缓冲器，触发器，加法器等。这些类型通常可以通过cell命名规则中看出来。Sram就不太一样了。前仿过程中，我们也用sram，但是用的是sram的fast_func模型（在命名上有fast_func后缀），后仿过程中要将其替换为指定corner下的，带时序检查的sram模型。如果不知道路径，直接找设计、或者后端要。

6.加入后仿仿真参数以及必要的文件加载（基于VCS）。因为后仿环境一般都是继承前仿环境而来，所以需要在仿真命令中加入后仿相关参数（编译参数）。主要有：

(1)后仿真参数，+neg_tchk, -negdelay, -sdfretain, -sdfverbose, 具体含义自己查一查。

(2)Sdf加载: -sdf max:test:sdf_path, 这里test是我们的模块名, sdf_path是对应的sdf文件路径。需要说一下的是max需要和用于setup检查的sdf匹配, min需要和用于hold检查的sdf匹配。

(3)无复位端寄存器初始化: 编译参数加+vcs+initreg+random, 仿真参数加+vcs+initreg+0。

(4)Cell单元的debug功能, 最好打开对cell单元的波形debug功能, 后仿的时候用得到, -debug_region+cell+encrypt。

(5)三级同步器的第一级触发器时序检查屏蔽: +optconfigfile+mask_file, 其中mask_file是符合vcs要求的触发器屏蔽代码。

(6)网表文件替换: rtl文件替换成网表文件, 这个简单, 但如果是在在rtl代码中局部替换部分harden为网表, 就需要使用vcs的libmap功能。搜索一下, 也很简单。

(7)Sram和标准单元路径加载: 上面找到了对应的路径, 这里当作普通文件吃进去就好了。

7. 什么是芯片 Bring-up

来源: <https://mp.weixin.qq.com/s/3zGSqLRCBnhgqDYyaJ6Etw>

作者: DMXAI

update 2026/08/16 03 : 40

芯片 Bring-up 是 Tapeout 之后, 团队基于真实硅片、封装、板卡和底层软件, 完成首次系统启动、状态确认和故障定位的过程。它的目标不是立即证明芯片完全正确, 而是先建立一个可供控制、观察和继续验证的基础环境。

7.1 Bring-up 发生在什么阶段?

芯片研发通常会经历架构定义、RTL 设计、流片前验证、综合实现、物理设计、Tapeout、晶圆制造、封装测试、板级集成、硅后验证和量产导入。Bring-up 发生在芯片回片早期。此时团队面对的已经不是仿真模型、Emulation 平台或 FPGA Prototype, 而是真实硅片。

在仿真环境中, 工程师可以查看大量内部波形, 逐拍分析信号变化。FPGA Prototype 能够提前运行部分系统路径和软件栈, 但在频率、电气特性、PHY、功耗和工艺行为上, 仍然与真实芯片存在明显差异。到了 Bring-up 阶段, 设计假设会第一次同时受到真实物理条件的检验。电源质量、参考时钟、封装连接、PCB、温度、噪声、工艺偏差和软件初始化顺序, 都可能影响芯片是否能够正常启动。

7.2 Bring-up 和其他验证活动有什么区别?

芯片项目中, 流片前验证、FPGA Prototype、Bring-up、硅后验证和量产测试经常被放在一起讨论, 但它们关注的问题并不相同。

活动	典型阶段	主要回答的问题
流片前验证	Tapeout 前	RTL 和 SoC 实现是否符合规格
FPGA Prototype / Emulation	Tapeout 前	软件栈和系统路径能否提前运行
Bring-up	回片早期	真实芯片能否进入可控、可观察、可运行状态
硅后验证	回片后	芯片的功能、性能、功耗、兼容性和可靠性是否达到目标
量产测试	流片前规划和开发，晶圆测试、封装测试和量产阶段使用	如何快速、稳定地筛选芯片，并控制覆盖率、良率和测试成本

Bring-up 通常是硅后验证的起点。它不会一开始就覆盖所有性能和可靠性指标，而是先打通继续验证所需的基础条件。如果 JTAG 无法访问、BootROM 没有执行、片上 SRAM 不可用，或者 DDR 初始化完全失败，后续的接口验证、性能测试和系统软件调试都无法正常展开。不同公司的流程划分可能略有差异，但 Bring-up 的核心任务基本一致：先让真实芯片进入一个可以继续工作的状态。此文DMXAI芯片验证原创，未授权转发。

7.3 Bring-up 通常怎样展开？

真实项目中的 Bring-up 很少完全按照流程图推进，但整体上都会遵循一条相似的主线：从风险最低、最容易观察的部分开始，逐步建立对芯片状态的信任。

7.3.1 确认电源和板级状态

Bring-up 通常从电源开始。工程师需要检查各条电源轨的电压、上电顺序和电流曲线，观察是否存在异常漏电、短路或局部过热。这个阶段常用到电源分析仪、万用表、示波器和板级控制脚本。

在确认基本状态之前，团队通常不会直接让芯片进入全速运行。先在受控条件下观察电流、温度和复位状态，可以降低损坏芯片或板卡的风险。

7.3.2 检查时钟和复位

电源状态基本可信后，下一步通常是确认参考时钟、PLL 和复位流程。PLL 是否锁定，复位是否在正确条件下释放，各个时钟域是否进入预期状态，都会影响芯片能否执行第一条有效指令。很多看起来像软件卡死的问题，最后会回到时钟或复位上。例如某个电源域尚未稳定，相关模块的复位却已经解除；或者参考时钟出现抖动，导致 PLL 偶发失锁。

7.3.3 打通调试入口

JTAG、SWD、UART 和 trace 接口是 Bring-up 阶段最重要的入口。JTAG 无法连接时，问题未必出在 TAP 控制器本身。电源域、复位、管脚复用、板级连接和安全配置都需要排查。一旦调试入口可用，团队就可以读取芯片 ID、检查状态寄存器、访问片上存储，并逐步确认芯片内部已经运行到哪个阶段。

7.3.4 建立最小执行路径

调试入口可用后，下一步是确认 BootROM、片上 SRAM 和最小固件是否能够执行。启动模式是否正确，strap 或 eFuse 配置是否生效，异常向量是否有效，片上存储能否读写，这些问题共同决定芯片能否从静态硬件状态进入软件控制状态。

串口打印出第一行日志，通常是一个重要节点，但距离完整启动还很远。它至少说明处理器、部分时钟、复位、存储和 UART 路径已经具备基本工作条件。

7.3.5 打通启动介质、DDR 和基础外设

最小代码能够执行后，团队才会进一步处理片外 Flash、DDR 或 HBM、内存映射、中断控制器和基础外设。DDR training 通过并不代表存储子系统已经完全稳定。后续还要验证不同地址范围、访问宽度、突发传输、DMA、缓存一致性和长时间压力。GPIO、I2C、SPI、UART、timer 等基础外设通常会较早进入测试，因为它们可以帮助团队建立更多观测和控制手段。此文DMXAI芯片验证原创，未授权转发。

7.3.6 启动系统软件和关键数据路径

基础硬件路径稳定后，Bring-up 才会逐步进入 bootloader、RTOS 或 Linux 阶段。此时需要确认 device tree、内存映射、中断路由、驱动初始化和固件配置是否与真实硬件一致。之后再逐步打开 PCIe、Ethernet、USB、MIPI、SerDes、NPU、DSP 或视频处理链路等关键模块。

Bring-up 不要求所有模块一次性全部工作。更常见的做法是逐项启用，每增加一个子系统，就确认它没有破坏此前已经稳定的路径。

7.4 为什么 Bring-up 难？

Bring-up 最麻烦的地方，是一个简单现象可能对应完全不同的根因。例如：串口没有输出，可能是 BootROM 根本没有执行，也可能是时钟或复位异常，还可能只是管脚复用、波特率或板级连接配置错误。

在仿真环境中，这些状态通常可以通过波形区分；真实芯片默认没有这样的可见性。工程师只能依赖电流变化、JTAG 状态、启动进度码、错误寄存器和有限的 trace，一层层缩小范围。

问题跨层也是常态。PCIe 无法枚举时，根因可能来自控制器状态机、SerDes、参考时钟、复位、寄存器配置或主机兼容性。DDR training 失败，也可能与 PHY 参数、板级走线、供电、温度、固件配置和初始化顺序有关。这类问题很难由单个团队独立闭环。硬件、验证、固件、驱动、板级、封装、电源和测试团队通常需要同时参与。

另一个难点是问题不一定稳定复现。有些故障只会在特定电压、温度、频率或负载下出现；有些问题与上电间隔、初始化顺序或前一次运行状态有关。跑通一次只能说明在某次条件下成功，不能说明路径已经稳定。

因此，Bring-up 不只追求“能跑”，还要建立可复现、可记录、可解释的证据链。

7.5 Bring-up 的准备工作要在回片前完成

Bring-up 虽然发生在回片后，但调试能力往往在芯片设计阶段就已经决定。如果芯片缺少状态寄存器、错误记录、启动进度码、恢复模式和关键路径的观测能力，那么回片后即使问题出现，团队也很难快速判断故障范围。

7.5.1 可观测性

- 关键模块的状态寄存器；
- sticky error bit 和错误码；
- BootROM 和固件启动进度码；
- trace、事件计数器和性能计数器；
- 总线错误、中断状态和超时记录。

7.5.2 可控制性

- 安全启动模式和恢复模式；
- 最小启动路径；
- 对时钟、复位和电源域的调试控制；
- 可选择的启动介质和启动配置；
- 可在异常状态下保留的诊断信息。

7.5.3 测试与诊断能力

- MBIST、LBIST、scan 等 DFT 机制；
- 固件诊断接口和寄存器访问工具；
- 板级自动化脚本；
- 可复用的启动测试；
- 日志收集和结果归档机制。

这些能力平时并不显眼，回片后却直接决定问题是几小时收敛，还是几周都停留在猜测阶段。

一个有效的调试设计，能够把“系统不工作”缩小为“某个电源域未进入状态”“某条总线访问超时”“某个启动阶段没有完成”或“某个 IP 报告协议错误”。范围缩小，才是真正的调试效率。

7.6 FPGA Prototype 能提前暴露哪些问题？

FPGA Prototype 无法复现真实硅片的 PLL、SerDes、DDR PHY、功耗、工艺和封装效应，但可以提前打通大量软件与系统路径。在回片前，团队可以通过原型平台验证：

- 启动流程是否完整；
- 固件和驱动对寄存器模型的理解是否正确；
- 中断、DMA 和内存映射是否能够跑通；
- Linux bring-up 是否存在明显阻塞；
- 基础运行负载是否能够执行；
- 日志格式、调试脚本和诊断工具是否可用。

这些工作不能消除真实硅片风险，但可以减少 Bring-up 阶段同时存在的未知数。如果启动流程、驱动初始化和基础软件路径已经在 FPGA Prototype 上跑通过，回片后遇到问题时，团队可以优先排查真实硅片特有的电气、时钟、PHY、封装和板级因素，而不必重新从整个软件栈开始怀疑。

最后总结一下，芯片 Bring-up 是在回片早期围绕真实硅片、板卡和底层软件展开的首次系统启动与故障定位。它通常从电源、时钟和复位开始，随后打通调试入口、最小执行路径、存储、基础外设、操作系统和关键数据链路。每完成一个阶段，团队就多一组可信状态，也能把后续问题限制在更小的范围内。

Bring-up 暴露的不只有 RTL 缺陷。电源、时钟、封装、PCB、固件、驱动、验证覆盖和调试规划，都可能在这个阶段接受检验。一次 Bring-up 的结果也会反过来影响下一版芯片：哪些状态需要增加观测点，哪些启动步骤需要加入进度码，哪些软件和系统路径应该提前放到 FPGA Prototype 上验证。

芯片能跑起来，只是第一步。能够稳定复现问题、快速缩小范围，并为后续硅后验证和软件开发提供可靠环境，才说明 Bring-up 真正取得了进展。

8. 【验证专项】寄存器专项测试

来源：<https://mp.weixin.qq.com/s/7OIzXLRCNamOKvddR2BbqQ>

update 2026/08/16 14 : 46

8.1 寄存器复位值测试

- 最优测试方案

全局/模块复位后，通过总线前门遍历读取所有寄存器，与Spec标准复位值逐一对标校验。

- 覆盖范围

异步复位、同步复位、断电保持寄存器初值；杜绝上电随机X态、复位值异常、寄存器初始化错误问题。

8.2 寄存器属性测试

- 最优测试方案

摒弃冗余随机激励，采用**固定极值特征图案**高效全覆盖：全0、全1、0x55/0xAA交替图案、单bit行走图案。
针对性校验所有寄存器字段属性：

1. RW读写字段：正常读写、字段掩码有效、无越权修改。
2. RO只读字段：软件写无效，数值保持不变。
3. 特殊控制位：W1C/W0C/W1S/W1T标准语义校验。
4. Reserved保留位：写忽略、读符合Spec定义。

- 覆盖范围

寄存器字段功能异常、掩码失效、保留位可篡改、特殊位动作逻辑错误、读写权限异常。

8.3 地址粘连测试

- 最优高效组合方案（提速+全覆盖无漏测）

March C- 9步三九算法工业通用最优方案，兼顾效率与全覆盖，无需11步冗余校验，完整覆盖：单/多地址线桥接短路、地址译码重叠、跨地址误写污染、双向时序敏感地址故障。

- 覆盖范围

地址线单bit粘连、多线桥接短路、地址译码冲突、地址双向访问时序故障、总线地址误选通污染。

8.4 数据粘连测试

- 最优测试方案

March C- 9步三九算法（寄存器数据粘连唯一高效全覆盖用例）

通过升序+降序双向遍历、多轮0/1互补读写翻转，强制激活寄存器数据位所有电平状态，100%检出**数据位 stuck0/stuck1固定粘连故障**。

- 关键区分

棋盘、行走位、普通遍历**不覆盖数据粘连**，仅March系列算法可同时覆盖地址+数据双粘连故障。

- 覆盖范围

寄存器DFF数据位固定0、固定1卡死，数据存储单元失效，读写数据固化异常。

8.5 地址边界测试

- 最优测试方案

仅测试**核心边界地址**，摒弃无效冗余地址，最高效覆盖边界风险：首地址、末地址、跨4K边界、字对齐/非对齐边界、寄存器块偏移极值地址。

对边界地址执行标准读写+0/1极值翻转校验，搭配非法边界地址访问校验总线响应。

- 覆盖范围

边界地址译码失效、边界地址读写异常、跨边界地址串扰、非对齐边界访问报错异常、非法边界地址污染有效

寄存器。

8.6 全地址遍历测试

- 最优测试方案

采用**分层轻量化遍历**，拒绝无脑全量遍历耗时（选择遍历地址页签和边界）：

1. 日常回归：基础全地址遍历+棋盘预筛，快速确认所有地址可达性；
2. 版本签核：基于March C- 9步完成全地址遍历校验，一举兼顾可达性、地址粘连、数据粘连，无需重复遍历。

- 覆盖范围

所有寄存器地址映射有效性、死地址、未绑定地址、地址遗漏、单地址通路失效问题。

9. [验证专项][UPF 功耗仿真]UPF 功耗仿真原理通俗版讲解

来源：https://mp.weixin.qq.com/s/OglwO_7oTdp9KEwmW1ss7w

update 2026/08/16 23 : 00

9.1 先搞懂普通仿真的问题

普通 Verilog/SV 仿真，相当于默认**整个芯片全程一直通电、电压不变、永远正常工作**：

- 不管真实芯片会不会关掉某块模块电源、进入睡眠，仿真里这块模块也一直在跑，信号都是正常 0/1。
- 不知道断电后电路会浮空、产生未知信号 X，也不知道隔离 / 保持寄存器的特殊行为。
- 没法验证睡眠唤醒会不会出错、会不会把坏信号扩散到整个芯片，也没法算出真实功耗。
- UPF 就是一份单独的 Tcl 说明书：告诉 EDA 仿真软件 —— 芯片电源是怎么划分、怎么开关、怎么防 X 污染、怎么保存状态、有哪些合法供电模式。

9.2 UPF 核心作用：给仿真器加上「电源模型」

9.3 第一步：划分电源区域 UPF 用 `create_power_domain` 把芯片分成好几块区域：

- 常电区域（AON）：比如电源管理电路、隔离单元、备份保持寄存器，永远有电，全程正常工作。
- 可断电区域：比如 UART、外设模块，可以整体关掉主电源省电 同时画出对应的电源线（supply net），还有专门给备份寄存器用的常电电源线。

第二步：加电源开关模型 `create_power_switch` 定义睡眠控制信号（sleep_en）：

- sleep_en=0: 可断电区域正常供电, 信号正常 0/1, 正常运行。
- sleep_en=1: 可断电区域主电源关掉 → 仿真器把这个区域内部信号标记为 **X 未知态 (模拟真实浮空 / 失效状态)**。

第三步: 隔离单元模型 (防 X 扩散) `set_isolation` 配置常电隔离单元和隔离使能信号 iso_en:

- 正常模式: 隔离关闭, 信号正常互通。
- 准备断电前: 打开隔离, 强制把可断电区域对外输出信号固定成 0/1, 不让内部 X 信号跑到主芯片里污染正常电路。
- 关键点: 隔离单元放在常电区域, 本身不会跟着外设一起断电, 持续生效。

第四步: 保持寄存器模型 (保存关键数据) `set_retention` 配置常电备份寄存器和保存信号 ret_save:

- 断电前: 把主寄存器数据复制到常电备份寄存器。
- 主域断电: 主寄存器变成 X, 备份寄存器由常电持续供电、保留原有数据。
- 上电唤醒: 把备份数据恢复回主寄存器, 恢复原来状态, 避免数据丢失、重头初始化。

第五步: 电源状态表 PST

`add_power_state` 定义合法供电模式 (正常 1V、睡眠低压、深度断电 0V) 仿真器会监控电源跳转, 不允许出现非法电压 / 非法上电顺序, 避免模拟芯片硬件损坏风险。

9.4 功耗统计原理

- 动态功耗: 仿真记录每根信号翻转的时刻和频率, 结合工艺库参数, 算出开关翻转消耗的功耗。
- 静态漏功耗: 根据当前电源模式 (正常 / 睡眠 / 断电) 和单元漏电模型, 计算晶体管漏电功耗。
- 睡眠模式下大部分模块断电, 漏功耗显著下降, 和真实芯片省电行为保持一致。

9.5 PA 功耗仿真整体运行机制

- 仿真器同时载入: 原始 RTL / 网表 + UPF 功耗规则 + 标准单元库模型。
- 正常运行时: 和普通仿真一样跑逻辑。
- 进入睡眠流程: 按 UPF 规则先开隔离、保存状态、再关闭子域电源, 子域内部信号变为 X、隔离挡住 X 扩散、备份寄存器保存状态。
- 唤醒流程: 子域重新上电、恢复备份状态、确认稳定后关闭隔离, 恢复正常通信。
- 全程校验时序和电源跳转规则, 同时统计整体功耗。
- 和纯功能仿真最大区别: **仿真器感知电源状态变化, 真实模拟断电 X 行为、隔离钳位、状态备份恢复和功耗变化, 不再默认全芯片永久上电。**

9.6 一句话总结

UPF 功耗仿真 = 用 UPF 脚本教会仿真软件识别芯片的多电源架构和低功耗策略，模拟模块断电浮空 (X)、隔离防污染、保持存状态、睡眠省电的真实行为，同时算功耗，验证低功耗流程是否正确，不再跑一个“永远满电”的假仿真。